APRESENTAÇÃO DA ATIVIDADE 3

Contêineres, namespaces e cgroups

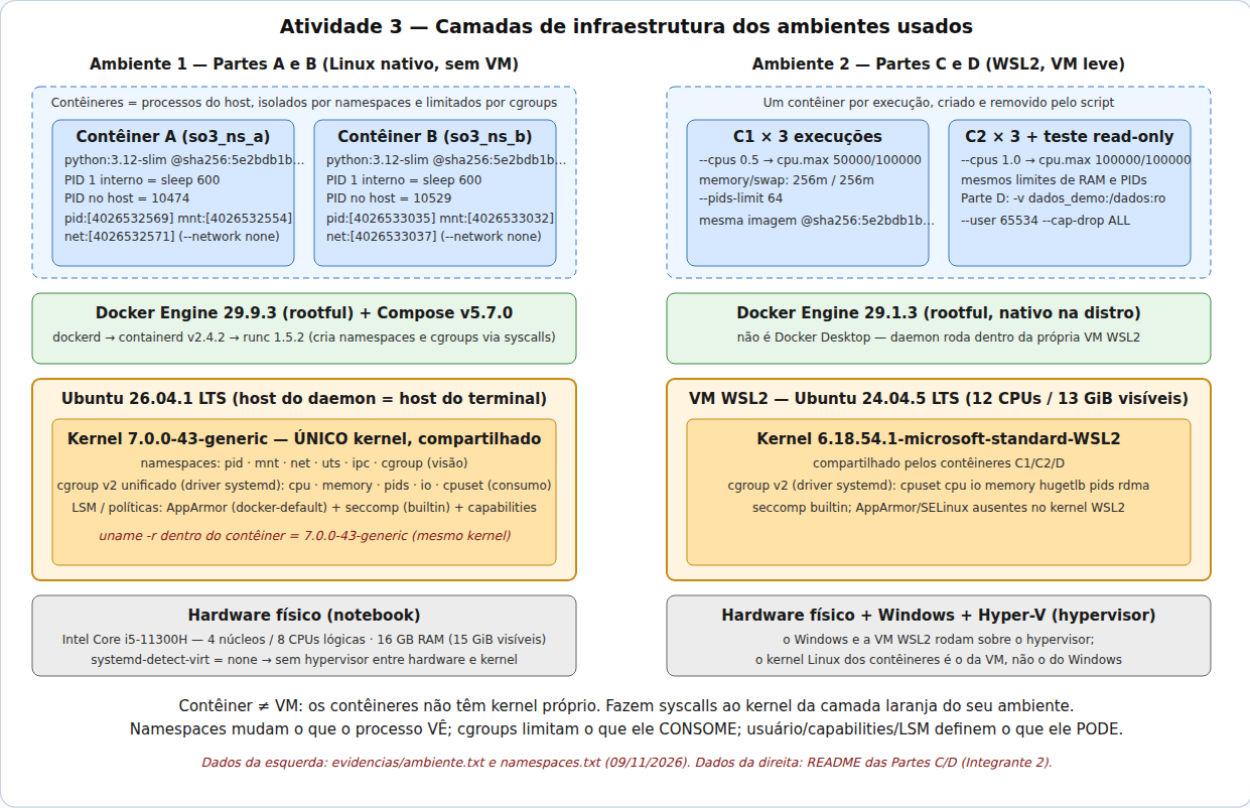
O que o contêiner vê e o que ele pode consumir, sem virar uma máquina virtual

Universidade Federal de Sergipe • Sistemas Operacionais • Novembro de 2026

Equipe: Anderson • Antônio Carlos • Antônio Martins • Davi • João Felipe • José Gustavo • Kaique

AMBIENTE · ANTÔNIO MARTINS

Contêiner não é máquina virtual



7.0.0-43

uname -r dentro do contêiner: o mesmo kernel do host

WSL2

O kernel compartilhado é o da VM, não o do Windows

Handles de namespaces

Namespace	Contêiner A	Contêiner B	Host
PID	4026532577	4026533042	4026531836
mount	4026532554	4026533039	4026531832
network	4026532579	4026533044	4026531833

- Todos diferentes: PIDs, montagens e rede separados
- Mesmo namespace só se o handle for idêntico (namespaces(7))
- Conferido de dentro e de fora do contêiner (root): 6 de 6 handles iguais

Provam visões separadas, não segurança absoluta nem limite de consumo.

Por que os dois têm PID 1?

sleep

/proc/1/comm em A e em B: dois processos, ambos PID 1

43568 · 43625

PIDs de A e de B no host

43568 1

NSpid: um processo, dois números

- Um namespace de PID novo reinicia a numeração
- O primeiro processo criado nele vira o init do namespace
- A unicidade do PID vale dentro de cada namespace
- Cada PID 1 é filho de um containerd-shim próprio (43543 e 43601)

Como medimos os cgroups

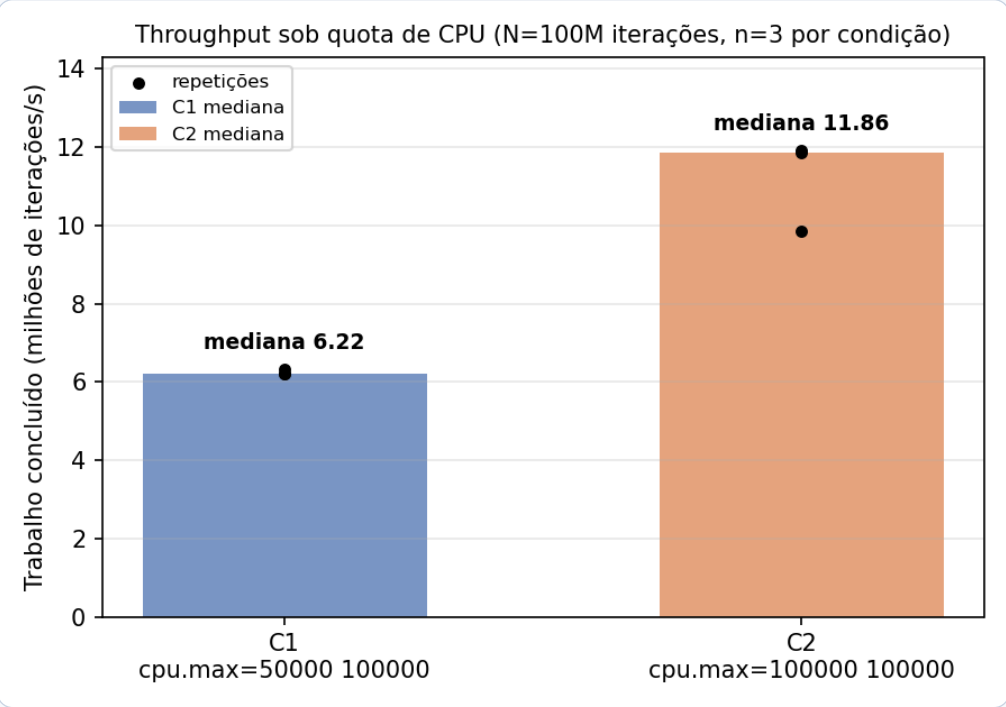
0,5 CPU
Condição C1

1,0 CPU
Condição C2

100 M
Iterações por execução, 3 repetições por condição

- Carga: soma de i^2 módulo $2^{61}-1$, uma thread, checksum conferido
- Iguais nas duas: 256 MiB, 64 PIDs, sem rede, usuário não root
- Limite confirmado lendo cpu.max dentro do cgroup

C1 × C2: trabalho por segundo



6,22

C1 (0,5 CPU): Mi iterações por segundo, mediana

11,86

C2 (1,0 CPU): Mi iterações por segundo, mediana

1,905

Razão C2/C1: perto de 2, mas não exata

Throttling: 162 de 162 períodos em C1 · nr_throttled = 0 em C2

Escrever em /dados: três casos

EROFS (30)
D1: nobody, mount ro

EROFS (30)
D2: root sem capabilities, mount ro

EACCES (13)
D3: nobody, mount rw

- A leitura funcionou nos três casos
- SHA-256 e listagem do host idênticos antes e depois
- CapEff = 0, no-new-privileges e seccomp ativos

Quem bloqueia o quê

Mecanismo	O que controla	Evidência
Flag ro	Gravação por ponto de montagem	EROFS até para o root (D2)
Permissões (UID, GID, modo)	Acesso por identidade	EACCES em D3
Capabilities, seccomp, no-new-privileges	Operações privilegiadas	Sem CAP_SYS_ADMIN, não remonta como rw
Namespace de mount	O que fica visível	Não define o que é gravável

Não prova isolamento absoluto: o kernel é compartilhado e o conteúdo continua legível.

Importância em infraestrutura compartilhada

Quotas

CPU, memória e PIDs: um job não monopoliza o servidor e o OOM fica no cgroup

Somente leitura

Corpus, modelos e pesos protegidos contra alteração e envenenamento

Menor privilégio

Não root, cap-drop ALL e no-new-privileges

- ro não protege a confidencialidade: os dados continuam legíveis
- Kernel compartilhado, cache L3 e banda de memória seguem expostos
- memory.max não limita a VRAM da GPU

Repositório e uso de IA

README

Preparação, reprodução, coleta e limpeza

Digest

Imagem python:3.12-slim fixada nos dois ambientes

Logs brutos

Scripts, CSV, gráfico e evidências no repositório

- Claude Code: scripts das Partes C e D, adotados após execução real
- Claude: revisão e redação, com valores conferidos nos logs
- Nenhuma saída de IA foi apresentada como medição

github.com/antoniomartins13/SO_UFS_2026_2_Mascarenhas_Antonio_Alves_Jose_AV3

Quem controla o quê

Namespaces	Cgroups	Permissões	Capabilities
O que o processo vê	Quanto ele consome	O que ele pode acessar	Que operações privilegiadas faz

- 0,5 CPU: throttling em 100% dos períodos; 1,0 CPU: mediana 1,91 vez maior
- Camadas complementares: não fazem do contêiner uma VM
- Limitações: WSL2, 3 repetições e carga sintética